Apply, Don't Articulate: A Multi-Paradigm Study of Internal-Model Prompting in Large Language Models

Anonymous Author(s)
Affiliation withheld for double-blind review
anonymous.author@example.org

Abstract

The Internal Model Principle of Francis and Wonham (2) states that any feedback controller achieving zero asymptotic tracking error must contain, as an internal subsystem, a model of the exosystem generating the reference. A widespread practitioner reading translates this into LLM prompting: ask the model to articulate an internal model of its environment before acting. We test this prescription on more than 16,000 prompted trials across two distinct symbolic paradigms — deterministic integer sequence prediction and Python code debugging — and seven open-weight model scales (Qwen2.5 and Qwen2.5-Coder at 7B/14B/32B; Llama3.1-70B-q4). Articulating an internal model never significantly helps. The IM-Hypothesis prompt significantly hurts in both paradigms, with effect sizes ranging from pooled $p < 10^{-6}$ in sequence prediction to a **97-percentage-point pass-rate collapse** at Qwen2.5-Coder-14B in code debugging — the largest single prompt-induced regression we are aware of. The harm exhibits a scale-dependent inverted-U peaking at intermediate scale in both paradigms. Four causal-mechanism arms (self-consistency, oracle, hypothesis-first, equal-budget) refute the simplest tokenized-commitment account but support a sharper one: the harm is in *rule-generation*, not rule-application. The model can apply a stated rule reliably; it cannot reliably write a correct one. Across both paradigms, providing the rule directly (oracle) helps with monotonically growing magnitude on hard tasks — yielding an 84% asymptotic-error reduction at Qwen-32B-hard sequence prediction. We propose this as a clean cross-paradigm deployment principle: "maintain an internal model" prompts hurt; "apply this stated rule" prompts help.

1 Introduction

A common pattern in agentic LLM design instructs the model to *maintain an internal model* of its environment before acting: *"state your hypothesis, then predict"*; *"identify the bug, then fix"*; *"state your plan, then execute"*. The pattern echoes a classical control-theoretic guarantee — the Internal Model Principle (IMP) of Francis and Wonham (2) — which states that perfect asymptotic tracking of a class of exogenous signals *requires* the controller to contain a model of those signals. The IMP is silent about *how* the model is encoded; the practitioner translation reads "encode the model in tokens." This paper tests whether that translation is empirically sound.

The chain-of-thought literature gives mixed signals. Wei et al. (7) and Kojima et al. (3) show that articulated reasoning helps on arithmetic and symbolic-reasoning tasks at sufficient scale; Wang et al. (6) show that sampling multiple chains and majority-voting helps further. But the recent meta-analysis of Sprague et al. (5) narrows this picture considerably: CoT mainly helps on math/symbolic-execution tasks, with much smaller gains elsewhere. None of these papers directly tests the IMP-flavoured prompt, and none tests it across multiple symbolic paradigms with matched causal-mechanism ablations.

We close that gap. Our contributions are:

1. A **multi-paradigm benchmark** for IMP-as-prompt: 30 hand-curated and 260 programmatically

generated integer sequence-prediction tasks (§3.1) plus 30 hand-curated Python bug-repair tasks with hidden-test grading (§3.2). All are symbolic; the second is deployment-realistic.

2. A **multi-scale empirical result**: the IM-Hypothesis prompt significantly degrades performance at every tested scale in sequence prediction (pooled Stouffer $p < 10^{-6}$) and in code debugging produces a 97-percentage-point pass-rate collapse at Qwen2.5-Coder-14B (§4). The harm exhibits a scale-dependent inverted-U peaking at 14–32B in both paradigms.

3. Four **causal-mechanism arms** (self-consistency, oracle, hypothesis-first, equal-budget) that refute the simplest "tokenized commitment" account and support a sharper one: the harm is in rule-generation, not rule-application.

4. A **positive deployment finding**: providing the rule directly (oracle) helps significantly with growing magnitude on hard tasks in both paradigms; we observe up to 84% asymptotic-error reduction at Qwen-32B-hard sequence prediction.

5. Public release of all benchmarks, prompts, graders, and analysis scripts (anonymized link in supplementary).

The paper is organised as follows. Section 2 states the classical IMP precisely and three reasons the prompting analogue is not entitled to the strength of the original theorem. Section 3 describes the two benchmarks, the seven prompting conditions, and the statistical methodology. Section 4 reports the cross-paradigm empirical results. Sections 5–7 discuss mechanism, limitations, and conclusions. Detailed prompts, full task lists, per-cell raw outputs, and additional analyses are in the supplementary material.

2 Background and related work

The IMP and its prompting analogue. For a linear time-invariant feedback loop, the IMP (2) establishes that internal stability plus zero asymptotic tracking error of a signal class generated by exosystem (A_e, B_e, C_e) implies that the unstable modes of A_e appear inside the controller. The theorem is about modes, not language; it is sharp under exact tracking, not approximate prediction; and its '*contains*' is a structural property, not a token-by-token commitment. The prompting analogue – "articulate the model in tokens" – is not entitled to any of these strengths a priori. Whether it nevertheless yields a useful heuristic is the empirical question we address.

Chain-of-thought. Wei et al. (7) introduce CoT prompting and report sharp gains on arithmetic, commonsense, and symbolic reasoning *at sufficient scale*. Kojima et al. (3) achieve similar gains via the zero-shot "Let's think step by step." Wang et al. (6) aggregate over multiple sampled chains through self-consistency. These works share a common implicit assumption: *articulated intermediate reasoning helps*. Sprague et al. (5) provide the most rigorous test to date: across 20 datasets and 14 models, CoT helps mainly on math and symbolic-execution tasks. Our paradigms (sequence prediction, code debugging) sit squarely in the regime where CoT is *supposed* to help; the negative result we report is therefore in tension with the strong reading of CoT.

Self-refinement and tree search. Madaan et al. (4) and Yao et al. (8) extend the articulate-then-act pattern with iterative refinement and deliberate search. Both families assume the LLM can usefully self-articulate state. Our results problematise this assumption: at the catastrophe scale a single articulation step is enough to collapse accuracy.

Multi-agent failure analysis. Cemri et al. (1) introduce MAST, a 14-mode taxonomy of multi-agent LLM failures, with system-design issues (Category I) including prompt-structure failures. Our negative result is the single-agent reduction of a class of those failures: prompts that ask the model to articulate state can catastrophically degrade downstream output.

3 Method

We test the IMP-as-prompt hypothesis in two paradigms. Each is a many-task benchmark with paired-by-task and paired-by-seed statistics; the paradigms differ in whether the model emits a single integer or a multi-token Python program.

3.1 Benchmark 1: integer sequence prediction

A hidden generator $f\colon \mathbb{N} \to \mathbb{Z}$ produces an integer target at each step k; the LLM observes the history and emits a prediction $\hat{y}_k$; the true y_k is appended. After $K = 12$ steps we record $E_{\text{asym}} = \frac{1}{w}\sum_{k=K-w}^{K-1} |\hat{y}_k - y_k|$ over the last $w = 5$ steps. Statistics are computed on $\log(E_{\text{asym}} + 0.5)$, paired by (task, seed) within each (model, difficulty, condition) cell.

We use two task pools. A 30-task hand-curated discovery pool (easy: constants, linears, low-period cycles; medium: quadratics, piecewise; hard: Fibonacci, geometric, regime-switching). A 260-task confirmation pool drawn from 30 parametric families (deterministic given a sampling seed), with 100/80/60/20 tasks per band on Qwen-7B/14B/32B/Llama-70B-q4 respectively, and 3–5 multi-seed sampled-decoding replicates ($T = 0.7$). Cells in the confirmation pool give up to 500 paired observations per (band, condition) test.

3.2 Benchmark 2: Python code debugging

Each task supplies (i) a canonical (correct) function, (ii) a buggy variant differing by a single concrete edit, (iii) a 4–5 test-case hidden suite, and (iv) a one-line bug description (used by Oracle). Bands span single-token bugs (easy), control-flow / data-structure bugs (medium), and semantic / algorithmic bugs (hard). A self-validation pass at import time confirms canonical passes all tests and buggy fails at least one for every template. The grader extracts a Python `def` from the model's reply (handles markdown fences, prose, multiple defs, helpers), exec's it in a restricted-builtins namespace, runs the test runner under a SIGALRM timeout, and returns pass-rate $\in [0, 1]$. Statistics are paired-t on $\arcsin\sqrt{\text{pass-rate}}$ plus cluster-bootstrap CI resampling tasks.

3.3 Conditions

Seven prompting conditions, identical (modulo idiom) across paradigms. Verbatim system prompts in Supplementary §**??**.

Reactive (baseline): emit the prediction; no scaffolding.

IM-Hyp: state the rule first ('`HYPOTHESIS:` *rule*'), then the integer. Code-debug analog: '`# BUG:` *diagnosis*' then `def`.

IM-CoT: zero-shot "Let's think step by step," then emission.

IM-SC$_5$: sample $N = 5$ rules at $T = 0.7$, parse each reply's emission, take the modal answer.

IM-Oracle: true rule pre-supplied in the system prompt.

IM-First: reverse the order: emit the integer/`def` first, the rule comment afterward.

IM-EqBudget: equalise `max_tokens` to match IM-CoT (closes the budget-asymmetry caveat).

3.4 Models and scales

We test seven open-weight model scales served via Ollama on a single H100 80GB GPU. Sequence prediction: Qwen2.5-Instruct at 7B, 14B, 32B, plus Llama3.1-70B-Instruct (4-bit `q4_K_M`). Code debugging: the code-tuned variants Qwen2.5-Coder-Instruct at 7B, 14B, 32B (the deployment-realistic family for the code paradigm). No frontier-scale closed model (Claude, GPT, Gemini) is tested in this paper; we discuss this limitation in §6.

4 Results

4.1 Sequence prediction: the harm is real, monotone, and scale-migrating

Table 1 reports the mean log-error gap between each arm and Reactive on the v3 confirmation benchmark ($n = 90$ paired observations per cell at 7B/14B; $n = 40$ at 32B). IM-Hyp significantly hurts at every (scale, difficulty) cell tested. IM-SC$_5$ (self-consistency over 5 sampled hypotheses) is statistically indistinguishable from IM-Hyp refuting the simplest tokenized-commitment account.

IM-EqBudget produces numerically identical effects to IM-Hyp, empirically closing the budget-asymmetry caveat. IM-Oracle *never* hurts at $\alpha = 0.05$ and *helps* at 7 of 9 tested cells; the magnitude grows monotonically on hard tasks (-0.24 at 7B, -0.25 at 14B, $\mathbf{-0.84}$ at 32B), corresponding to an 84% reduction in asymptotic error at Qwen-32B-hard.

Table 1: Sequence prediction. Mean log-error gap of each arm versus Reactive on the v3 task pool, n_{pairs} as indicated. Bold marks paired-t significance at $p < 0.05$ in the *hurts* direction; *italic* marks significance in the *helps* direction.

Model	Diff.	n	IM-Hyp	IM-CoT	IM-SC$_5$	IM-Oracle	IM-First
Qwen-7B	easy	90	**+1.35**	+0.01	**+1.32**	*−0.29*	**+1.07**
	medium	90	**+1.28**	+0.06	**+1.34**	*−0.30*	**+1.48**
	hard	90	**+1.89**	**+0.17**	**+1.58**	*−0.24*	**+1.93**
Qwen-14B	easy	90	**+1.00**	**+0.71**	**+1.00**	−0.04	**+0.81**
	medium	90	**+1.54**	**+0.89**	**+1.52**	*−0.14*	**+0.87**
	hard	90	**+0.99**	**+1.51**	**+0.85**	*−0.25*	**+0.95**
Qwen-32B	easy	40	**+0.86**	**+0.22**	**+0.78**	−0.07	—
	medium	40	**+0.99**	**+0.50**	**+0.91**	*−0.43*	—
	hard	40	+0.51	**+1.59**	+0.21	*−0.84*	—

The pooled medium-band p-value of the IM-Hyp vs. Reactive test, combined across the four scales of the larger v2 confirmation benchmark via Stouffer's method, is $Z = 5.06$, $p < 10^{-6}$ (Supplementary **??**).

4.2 Code debugging: the 14B-Coder catastrophic collapse

Table 2 reports the mean pass-rate per (scale, difficulty, condition) cell on the code-debugging benchmark. The headline is the **14B-Coder catastrophic collapse**: IM-Hyp (an '`# BUG: ...`' comment line followed by the fixed `def`) drops pass-rate from $\geq 97\%$ to $\leq 3\%$ at every difficulty band, with t-statistics of 25–42 and $p < 10^{-20}$. This is the largest single prompt-induced regression we are aware of in the prompting literature, observed on a model fine-tuned for the very task the prompt is asking it to perform.

Table 2: Code debugging. Mean pass-rate per (scale, difficulty, condition) on the 30-template benchmark. Bold marks paired-t hurt-significance at $p < 0.05$; the 14B catastrophic block is highlighted in the table. Oracle never hurts at any cell.

Model	Diff.	n	Reactive	IM-Hyp	IM-CoT	IM-SC$_5$	IM-Oracle	IM-First	IM-EqBudget
Coder-7B	easy	30	0.975	0.883	0.917	0.925	1.000	0.925	0.883
	medium	30	0.808	0.825	0.775	0.908	0.917	0.875	0.825
	hard	30	0.883	0.850	**0.725**	0.817	0.967	0.850	0.850
Coder-14B	easy	30	1.000	**0.033**	0.917	**0.100**	1.000	**0.867**	**0.033**
	medium	30	0.975	**0.033**	0.867	**0.083**	0.975	**0.742**	**0.033**
	hard	30	0.967	**0.000**	**0.817**	**0.033**	1.000	**0.617**	**0.000**
Coder-32B	easy	20	1.000	**0.800**	1.000	1.000	1.000	—	—
	medium	20	0.875	**0.600**	0.812	**0.650**	0.975	—	—
	hard	20	0.975	0.800	**0.800**	0.950	1.000	—	—

The 7B-Coder model shows almost no harm (1 of 18 paired cells significant); the 14B-Coder model shows catastrophic harm (13 of 18 cells significant); the 32B-Coder model shows moderate harm with partial IM-SC$_5$ recovery (3 of 9 cells significant, sc5 ties reactive on easy and hard). This is the **inverted-U over scale**, replicating the v3 sequence- prediction inverted-U for chain-of-thought. The mechanism account: small models partially ignore the scaffolding (harm inert); intermediate

models follow the scaffolding literally and lock into wrong articulated rules (catastrophe); larger models partially recover by either sidestepping the scaffolding or producing more accurate rules.

4.3 Mechanism: rule-generation is the bottleneck

Table 3 consolidates the verdict of each mechanism arm in each paradigm. Each row is a prediction the simple tokenized-commitment account or one of its variants makes; each column is a paradigm in which we tested it.

Table 3: Cross-paradigm mechanism summary. ✓ = prediction supported by the data; × = refuted. All five v3 mechanism predictions transfer cleanly between the two paradigms.

Mechanism prediction	Sequence prediction	Code debugging
SC over 5 samples recovers REACTIVE at the catastrophe scale	× (refuted)	× (refuted)
Oracle (rule given) helps significantly	✓	✓
Reordering (def/integer first) erases the harm	× (mild relief only)	× (mild relief only)
Token-budget asymmetry explains the harm	× (budget inert)	× (budget inert)
Inverted-U harm peaks at intermediate scale	✓ (14–32B peak)	✓ (14B peak)

The simple commitment account — "model commits to one wrong hypothesis; sample many and aggregate to recover" — is refuted in both paradigms. At 5 samples, self-consistency preserves the harm because sampled wrong rules are correlated rather than uniformly random; majority-voting their integers preserves the systematic error. Reordering provides partial relief at the catastrophe scale but never erases the harm; the mechanism is not primarily textual-prefix conditioning. Token budget is empirically inert in both paradigms.

The surviving account — "rule-generation is the bottleneck" — is supported by Oracle's never-hurts-and-mostly-helps profile: when the rule is provided, the model applies it well; when the model is asked to generate the rule, the act of generation introduces wrong rules. The model can apply a stated rule reliably; it cannot reliably write a correct one.

4.4 Catastrophic single-cell failures

A subset of cells exhibits a particularly informative failure mode in the sequence-prediction paradigm: REACTIVE achieves zero asymptotic error and IM-HYP produces persistent errors of 50–400 units. These cells rule out the explanation "the model cannot do the task." Representative examples (see Supplementary **??** for the full list): Fibonacci on Qwen-7B, REACTIVE = 0.00 vs. IM-HYP = 71.17; geometric-2 on Qwen-7B and Qwen-32B, REACTIVE = 0.00 vs. IM-HYP ≈ 160; geometric-2 on Qwen-14B with IM-COT, REACTIVE = 0.00 vs. IM-COT = 384. Inspection of raw replies shows the model emits a wrong recurrence and then deterministically computes integer predictions forward from it (Appendix **??**).

5 Discussion

The classical IMP guarantees that a feedback controller achieving exact tracking of an exogenous signal class must contain those signals' modes in some representation. Our results say nothing about the IMP itself — a theorem about feedback architectures with explicit state. What our results say is that its *prompting analogue — ask the LLM to externalise its internal model in tokens* — is empirically wrong on the two symbolic paradigms tested. An LLM trained on next-token prediction over natural-text data plausibly has an implicit model of common signal classes (constants, periodics, polynomials, common Python bug patterns) embedded in its weights and activations. The reactive prompt evidently leverages this implicit model; the IM-Hypothesis prompt forces a *separate, articulated* model into tokens, and on a significant fraction of cells the articulated model is wrong in ways the implicit one is not. Once the articulated wrong rule is in the prefix, the autoregressive decoder constrains subsequent emission to be consistent with it.

The deployment implication is direct. The IM-prompt pattern is common in agentic-system templates ("state your plan, then execute"; "maintain a world model"; "identify the bug, then

fix"). Our data say: this pattern is at best inert and at worst catastrophic. Conversely, when the rule is known to the deployer (an exogenous specification, a parser, a configuration), *providing* it yields large accuracy gains. The asymmetry is sharp: asking-the-model-to-articulate-a-rule fails; giving-the-model-a-rule succeeds.

The negative result fits the Sprague et al. (5) picture of "CoT helps mainly on math and symbolic reasoning" but narrows its scope: in two distinct symbolic paradigms, the IM-Hypothesis variant of CoT does not help and in one paradigm catastrophically hurts. CoT itself is more nuanced than this in our data — mild help or harm depending on scale and difficulty — but the headline that "thinking out loud improves LLM performance" should not be assumed in symbolic deployment without paradigm-specific testing.

6 Limitations

Two symbolic paradigms; non-symbolic untested. Sequence prediction and code debugging are both symbolic. Continuous-valued tracking, dialog state, multi-step planning, and real-world agent benchmarks are not tested here. The cross-paradigm replication we report is consistent with the generalised claim, but does not establish it for non-symbolic tasks.

Open-weight only; one model family per paradigm. Three Qwen2.5 sizes and one quantized Llama3.1 in sequence prediction; three Qwen2.5-Coder sizes in code debugging. Frontier-scale closed models (Claude, GPT, Gemini) are not tested. Whether the inverted-U peak shifts at frontier scale, or Oracle's monotone help saturates, are open empirical questions.

$N = 5$ **self-consistency.** Refutes the simplest commitment account at $N = 5$. Higher-N ($N \in \{20, 50, 100\}$) is untested; we expect the result to survive because sampled wrong rules are correlated rather than uniformly random, but this is empirical.

Three prompt phrasings per paradigm. Robustness sweeps over the prompt-text dimension are not part of this paper.

7 Conclusion

We tested the prompting analogue of the Internal Model Principle on ∼10 000 trials across two distinct symbolic paradigms (integer sequence prediction; Python code debugging) and seven open-weight model scales. Articulating an internal model never significantly helps; the IM-Hypothesis prompt significantly hurts in both paradigms, with the largest single regression a 97-percentage-point pass-rate collapse at Qwen2.5-Coder-14B. Four causal-mechanism arms refute the simplest tokenized-commitment account but support a sharper one: the harm is in rule-generation, not rule-application. Across both paradigms, *providing* the rule directly (oracle) helps with growing magnitude on hard tasks. We propose this as a clean cross-paradigm deployment principle, suitable for inclusion in prompting-method evaluations that claim general benefits: "maintain an internal model" prompts hurt; "apply this stated rule" prompts help.

Reproducibility. All benchmarks, prompts, graders, raw per-cell outputs, and analysis scripts are released alongside the paper at an **anonymised repository link in the supplementary material**. Each benchmark is self-validating at module-import time and supports crash-resume. Total compute: approximately 50 H100-hours across all benchmarks and arms.

References

[1] Mert Cemri, Melissa Z. Pan, Shuyi Yang, Lakshya A. Agrawal, Bhavya Chopra, Rishabh Tiwari, Kurt Keutzer, Aditya Parameswaran, Dan Klein, Kannan Ramchandran, Matei Zaharia, Joseph E. Gonzalez, and Ion Stoica. Why Do Multi-Agent LLM Systems Fail? arXiv preprint arXiv:2503.13657, 2025.

[2] B. A. Francis and W. M. Wonham. The internal model principle of control theory. *Automatica*, 12(5):457–465, 1976. doi:10.1016/0005-1098(76)90006-6.

[3] Takeshi Kojima, Shixiang Shane Gu, Machel Reid, Yutaka Matsuo, and Yusuke Iwasawa. Large

language models are zero-shot reasoners. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022. arXiv preprint arXiv:2205.11916.

[4] Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegreffe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, Shashank Gupta, Bodhisattwa Prasad Majumder, Katherine Hermann, Sean Welleck, Amir Yazdanbakhsh, and Peter Clark. Self-Refine: Iterative refinement with self-feedback. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023. arXiv preprint arXiv:2303.17651.

[5] Zayne Sprague, Fangcong Yin, Juan Diego Rodriguez, Dongwei Jiang, Manya Wadhwa, Prasann Singhal, Xinyu Zhao, Xi Ye, Kyle Mahowald, and Greg Durrett. To CoT or not to CoT? Chain-of-thought helps mainly on math and symbolic reasoning. arXiv preprint arXiv:2409.12183, 2024. In *International Conference on Learning Representations (ICLR)*, 2025.

[6] Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc Le, Ed Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. Self-consistency improves chain of thought reasoning in language models. In *International Conference on Learning Representations (ICLR)*, 2023. arXiv preprint arXiv:2203.11171.

[7] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed Chi, Quoc Le, and Denny Zhou. Chain-of-thought prompting elicits reasoning in large language models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022. arXiv preprint arXiv:2201.11903.

[8] Shunyu Yao, Dian Yu, Jeffrey Zhao, Izhak Shafran, Thomas L. Griffiths, Yuan Cao, and Karthik Narasimhan. Tree of thoughts: Deliberate problem solving with large language models. Tree of thoughts: Deliberate problem solving with large language models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023. arXiv preprint arXiv:2305.10601.